



Machine Learning on Snowflake

End-to-End ML — From Raw Data to Production

MAY 2026 · SESSION 1



Chris Caudill

Senior AI/ML Architect

Snowflake

PRESENT

Senior AI/ML Architect

Snowflake

End-to-end ML platform strategy, customer architecture, production ML enablement.

5 YEARS

Senior ML Engineer

Amazon Web Services (AWS)

Enterprise-scale MLOps, Fuels Price Optimization, Agentic Systems.

10 YEARS

Senior Data Scientist

ExxonMobil

Investigative NLP, Market Signal Detection, Summarization Modeling.

 SERIES

Session Series:

Machine Learning on Snowflake

A 4-part deep dive into production ML capabilities on Snowflake.



SESSION 1 · NOW

End-to-End ML Overview

Platform overview, production-grade ML, live demo.

SESSION 2

Feature Engineering & Feature Store

Entities, FeatureViews, point-in-time correctness.

SESSION 3

Training, HPO & Model Registry

Distributed training, HPO, registry RBAC.

SESSION 4

Deployment, Monitoring & MLOps

Model Serving, Observability, CI/CD.

 AGENDA

Today's Agenda



Part 1 — Context & Vision

- › Why predictive ML still drives massive value
- › The standard ML lifecycle & its challenges
- › Snowflake's unified ML platform overview
- › What "production-grade" means



Part 2 — Live Demo

- › Patient risk stratification (end-to-end)
- › Feature Store → Training → Serving → Monitoring
- › Fully automated retraining pipeline
- › Everything inside one Snowflake account

 PLATFORM

Unified Platform for Gen AI & ML

Today's session focuses on the **Machine Learning** layer of the Snowflake platform.



APP DEVELOPMENT

Snowflake Intelligence | Notebooks | Streamlit

GENERATIVE AI

Cortex AI

MACHINE LEARNING

Snowflake ML

AI INFRASTRUCTURE

Container Runtime | GPUs

DATA & MODEL GOVERNANCE

Snowflake Horizon | Model Registry

💰 BUSINESS IMPACT

Predictive ML Still Drives Massive Business Value

While GenAI captures attention, predictive ML remains the workhorse behind critical decisions.



Fraud Detection

Real-time transaction scoring prevents billions in losses annually across financial services.



Demand Forecasting

Predict inventory needs across thousands of SKUs and locations to optimize supply chains.

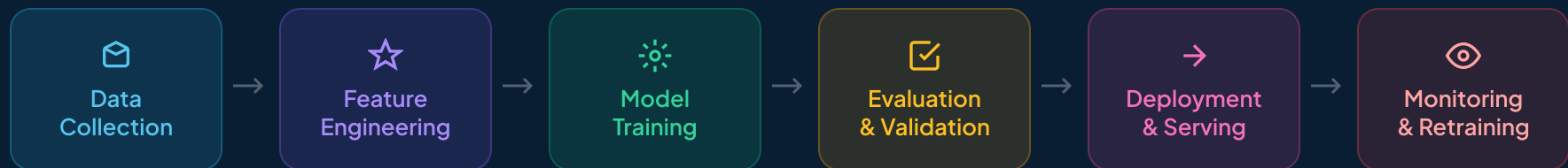


Risk Stratification

Classify patients, loans, or policies to allocate resources where they matter most.

 LIFECYCLE

The Standard Machine Learning Lifecycle



Each stage introduces complexity, tooling fragmentation, and data movement risk.

 THE PROBLEM

Traditional ML Pipelines Are **Fragmented**

Every stage requires a different tool, a different auth model, and another data copy.

Tool Sprawl

Airflow for orchestration, MLflow for tracking, SageMaker for training, Kubernetes for serving, Evidently for monitoring — each with its own config and billing.

Data Movement

Data is copied between warehouses, object stores, and compute clusters — introducing latency, cost, and governance gaps at every hop.

Security & Governance

Each tool has its own access model. No single pane of glass for who accessed what data to train which model.

Operational Burden

ML engineers spend more time managing infrastructure than building models. Production deployments take weeks, not hours.

Single Platform. No Data Movement. Unified Governance.

Everything your ML pipeline needs — inside one platform, under one security model.



No Data Movement

Training data, feature computation, model artifacts, and inference all happen inside Snowflake. Your data never leaves.



Unified Governance

RBAC, lineage, and data quality extend seamlessly from your data to your ML pipeline. One security model for everything.



Zero External Tools

Airflow, MLflow, SageMaker, Kubernetes — replaced by native Snowflake capabilities. One platform, one bill.



DATA



TRAIN



SERVE



PRODUCTION-GRADE

What "Production-Grade" Really Means

Every component is built to the standards required for real-world, business-critical workloads — not just prototypes or demos.



Scalable

Distributed training across N nodes. Auto-scaling endpoints 1→N replicas. Feature retrieval over 50M+ rows. Workloads grow without re-architecture.



Reliable

Quality gates block bad models. Automated retraining with built-in fallbacks. Your endpoint always serves a validated model — never a broken one.



Observable

Drift detection on features & predictions. SHAP explainability. Configurable alerting. You know when something is wrong before users do.



Functional

Feature Store, Model Registry, experiment tracking, versioned artifacts, REST serving — every ML lifecycle piece is first-class, not a bolt-on.



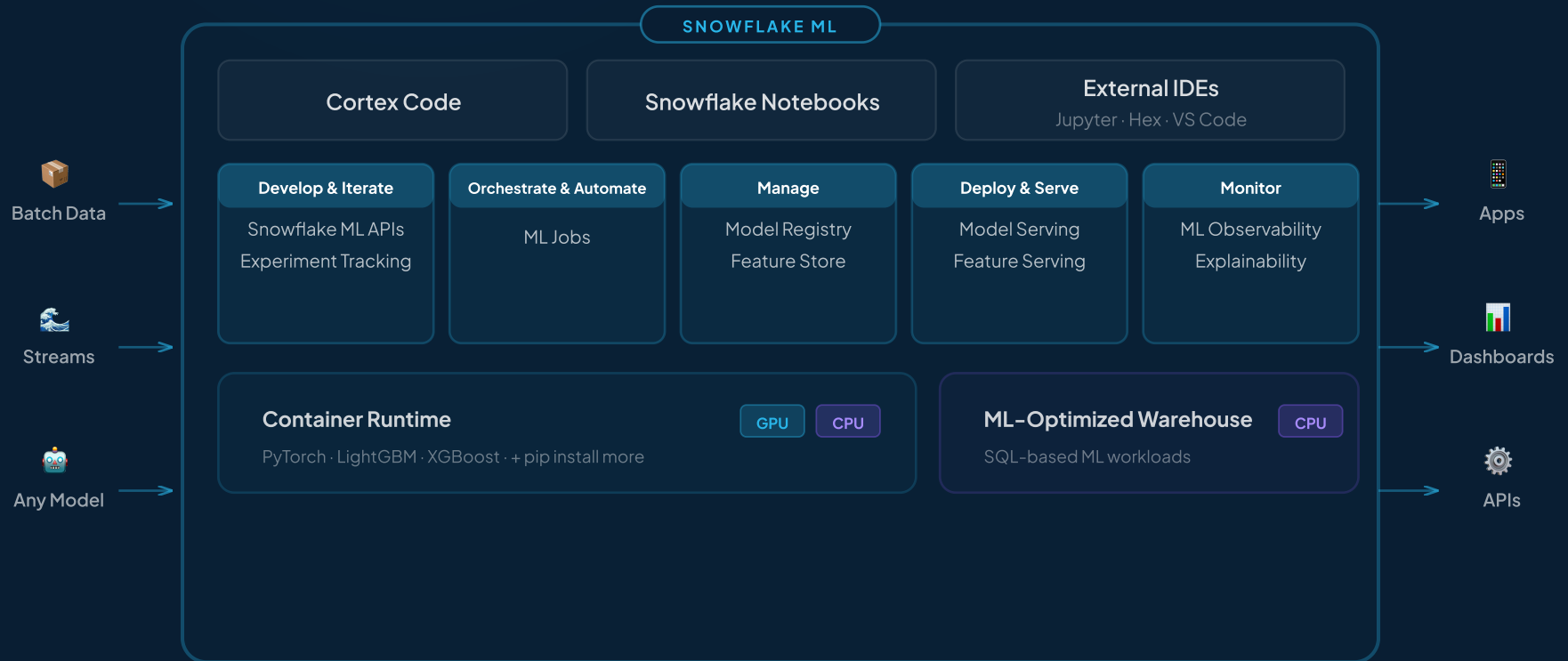
Governed

End-to-end lineage from source to predictions. RBAC on every object. Audit trails for compliance. One model for data + ML governance.

Production-grade = ready for the workloads that matter, from day one.

Snowflake ML: Comprehensive, Flexible, Integrated

Agentic ML tooling accelerates productivity for DS and MLEs



PyTorch, the PyTorch logo and any related marks are trademarks of The Linux Foundation.

Model Lifecycle in Snowflake

Migrate ML development from external IDEs — or build natively — with unified governance across every stage.



Develop & Iterate

- › Snowflake Notebooks
- › Distributed ML APIs
- › Cortex Code
- › Experiment tracking

Orchestrate

- › ML Jobs
- › Task DAGs
- › Remote Execution

Manage

- › Model Registry
- › Feature Store
- › Versioned artifacts

Deploy & Serve

- › Model Serving
- › SPCS REST Endpoints
- › Auto-scaling

Monitor

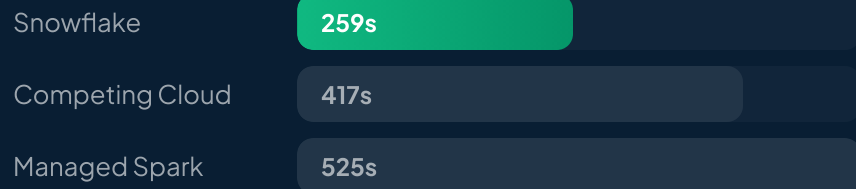
- › ML Observability
- › ML Explainability

UNIFIED GOVERNANCE ACROSS EVERY STAGE

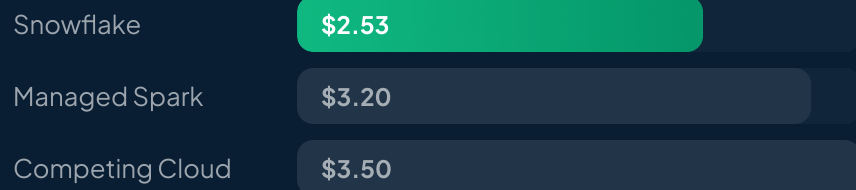
 PERFORMANCE

Advanced & Highly Scalable

TRAINING DURATION



TRAINING COST



SCALABILITY FEATURES

- ✓ Distributed training across N nodes — same API, just change `num_instances`
- ✓ GPU-accelerated scikit-learn via RAPIDS
- ✓ Partition many-model training: thousands of models in parallel in minutes
- ✓ Feature retrieval scales to 10M+ entities, 50M+ records with ASOF joins
- ✓ Auto-scaling REST endpoints: 1 to N replicas based on load

 LIVE DEMO

Today's Demo: Patient Risk Stratification



The Objective

Classify hospital patients into acuity levels based on clinical vitals and lab values.

CLASSIFICATION LABELS

CRITICAL

HIGH

MODERATE

LOW

WHY THIS MATTERS

Early stratification enables proactive care, reduces readmissions, and optimizes hospital resource allocation in real-time.



The Data

Clinical vitals and lab values for each patient encounter.

INPUT FEATURES

 Heart Rate

 Blood Pressure

 Respiration

 BMI

 Temperature

 Lab Values

ENGINEERED FEATURES

SHOCK_INDEX

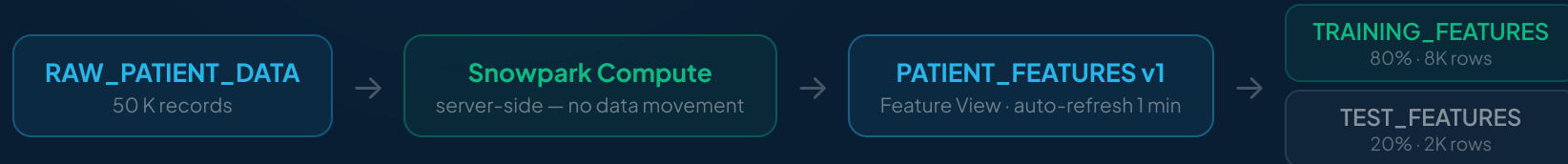
PULSE_PRESSURE

BMI_CATEGORY

VITAL_SIGNS_SEVERITY

Feature Engineering & Feature Store

Encode domain expertise once — reuse everywhere. Eliminates training/serving skew.



4 ENGINEERED BIOMARKER FEATURES



SHOCK_INDEX

Heart Rate ÷ Systolic BP

Numeric



PULSE_PRESSURE

Systolic – Diastolic BP

Numeric



BMI_CATEGORY

Underweight / Normal / Overweight / Obese

Categorical



VITAL_SIGNS_SEVERITY

Aggregate abnormal-vital score 0–9

Numeric

WHY THIS MATTERS

- Same formula at train time *and* serve time — zero skew
- Point-in-time correctness prevents data leakage
- Auto-refreshes every minute; inference uses fresh data

STEP1_FEATURE_ENGINEERING.PY

```
# 1. Initialize Feature Store
from snowflake.ml.feature_store import FeatureStore, CreationMode
fs = FeatureStore(session, database, schema,
                  default_warehouse=warehouse,
                  creation_mode=CreationMode.CREATE_IF_NOT_EXIST)

# 2. Register PATIENT entity
entity = Entity("PATIENT", join_keys=["PATIENT_ID"])
fs.register_entity(entity)

# 3. Build features server-side (Snowpark pushdown)
feature_df = df.with_column("SHOCK_INDEX",
                           F.col("HEART_RATE") / F.col("SYSTOLIC_BP"))
               .with_column("PULSE_PRESSURE",
                           F.col("SYSTOLIC_BP") - F.col("DIASTOLIC_BP"))

# 4. Register Feature View (auto-refreshes every 1 min)
fv = FeatureView("PATIENT_FEATURES", [entity],
                 feature_df, refresh_freq="1 minute")
fs.register_feature_view(fv, version="v1", block=True)

# 5. Point-in-time retrieval
training_df = fs.retrieve_feature_values(
    spine_df=spine.sample(frac=0.8), features=[fv])
```

Distributed Remote Training & Evaluation

Multi-node ML Job on SPCS. HPO params loaded from table. Training data never leaves Snowflake.

DISTRIBUTED TRAINING ARCHITECTURE

COORDINATOR
RANK 0
aggregates + registers

Worker
RANK 1

Worker
RANK 2

Each node receives **RANK** + **WORLD_SIZE=3** env vars from SPCS

MODEL EVALUATION RESULTS

87.3%
ACCURACY

86.4%
F1 MACRO

PROMOTION GATE — PASSED ✓

87.3% ≥ 80%
ACCURACY

86.4% ≥ 75%
F1 MACRO

STEP2_TRAIN.PY

```
# Submit multi-node distributed job via ML Jobs
from snowflake.ml.jobs import submit_directory
from snowflake.ml.registry import Registry

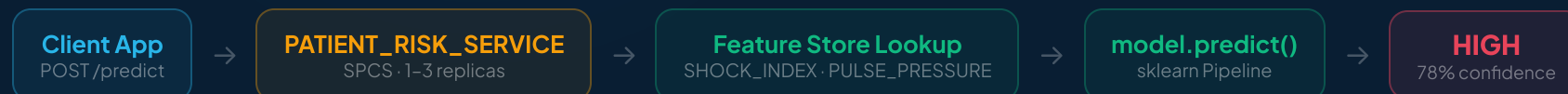
job = submit_directory(
    dir_path="source",
    entrypoint="train.py",
    compute_pool="ML_DEMO_COMPUTE_POOL",
    stage_name="@DB.SCHEMA.JOB_PAYLOADS",
    target_instances=3, # sets RANK + WORLD_SIZE
)
job.wait() # streams SPCS logs live

# Evaluate the newly registered version
registry = Registry(session)
mv = registry.get_model("PATIENT_RISK_MODEL").version(version_name)
metrics = mv.get_metrics()

# Promotion gate — must pass BEFORE deploy step
if (metrics["accuracy"] < 0.80
    or metrics["f1_macro"] < 0.75):
    raise RuntimeError("Model failed promotion gate")
```


REST Endpoint Deployment

Deploy in minutes. Auto-scale 1–3 replicas. Fully governed by Snowflake security.



SPCS SERVICE CONFIGURATION

Service Name	PATIENT_RISK_SERVICE
Compute Pool	ML_DEMO_COMPUTE_POOL
Replicas	1 min · 3 max (auto-scale)
Status	● RUNNING

SECURITY MODEL

- OAuth or key-pair auth — no API keys to manage
- Network policies and PrivateLink apply automatically
- Service VPC — never exposed to public internet

STEP4_DEPLOY.PY

```

# Get model version from Registry and deploy
from snowflake.ml.registry import Registry

registry = Registry(session,
                    database_name=db, schema_name=schema)

# Get the latest promoted version
mv = registry.get_model("PATIENT_RISK_MODEL").default

# One call deploys an auto-scaling SPCS service
mv.create_service(
    service_name="PATIENT_RISK_SERVICE",
    compute_pool="ML_DEMO_COMPUTE_POOL",
    min_instances=1,
    max_instances=3, # auto-scale
)

# Health check: run inference on sample rows
sample = session.table(test_table).limit(3)
predictions = mv.run(sample, function_name="predict")
# → DataFrame: output_feature_0 = "HIGH" ...
  
```

Model Monitoring & Drift Detection

Catch silent model decay before it impacts patient outcomes. No external observability tools.

Inference Logs

live predictions

**PATIENT_RISK_MONITOR**

snowflake.ml.monitoring

**TEST_FEATURES**

training baseline

**Drift Scores**

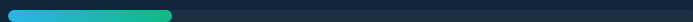
PSI per feature

**⚠ Alert**

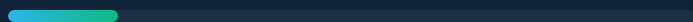
PSI > threshold → retrain

FEATURE DRIFT (VS. BASELINE)

SHOCK_INDEX PSI 0.12 ✓



PULSE_PRESSURE PSI 0.08 ✓



VITAL_SIGNS_SEVERITY PSI 0.31 ⚠



HEART_RATE PSI 0.09 ✓



⚠ ALERT TRIGGERED

VITAL_SIGNS_SEVERITY PSI 0.31 exceeds threshold 0.25.

Recommend: EXECUTE TASK to retrain on new distribution.

STEP5_MONITOR.PY

```
# Native model monitoring via snowflake.ml.monitoring
from snowflake.ml.monitoring import ModelMonitor
```

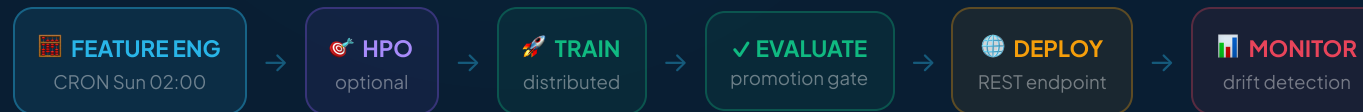
```
monitor = ModelMonitor.create(
    session=session,
    name="PATIENT_RISK_MONITOR",
    model_name="PATIENT_RISK_MODEL",
    model_version_name=version_name,
    source_table="HEALTHCARE.INFERENCE_LOGS_VIEW",
    baseline_table="HEALTHCARE.TEST_FEATURES",
    timestamp_column="TIMESTAMP",
    prediction_column_name="RISK_LEVEL",
    id_columns=["RECORD_ID"],
    warehouse="ML_DEMO_WH",
)
```

```
status = monitor.status
# → {"status": "ACTIVE", ...}
```

```
# Drift alert triggers retrain pipeline
monitor.setup_drift_alert(
    drift_metric="psi",
    drift_threshold=0.25,
    retrain_task="PIPELINE_FEATURE_ENG_TASK",
)
```

Automated Pipeline — Snowflake Task DAG

Each step is a Python Stored Procedure. Tasks chain them into a weekly CRON DAG.



DAG.PY

```

-- Wrap each step in a Python Stored Procedure
CREATE OR REPLACE PROCEDURE RUN_FEATURE_ENGINEERING()
RETURNS VARIANT LANGUAGE PYTHON
RUNTIME_VERSION = '3.11'
PACKAGES = ('snowflake-ml-python', 'scikit-learn')
IMPORTS = ('@JOB_PAYLOADS/source.zip')
AS $$ def handler(session):
    from source.pipeline.step1_feature_engineering import run
    return run(get_config(), session) $$;

-- Root task: CRON weekly schedule
CREATE OR REPLACE TASK PIPELINE_FEATURE_ENG_TASK
WAREHOUSE = ML_DEMO_WH
SCHEDULE = 'USING CRON 0 2 * * 0 America/Los_Angeles'
AS CALL RUN_FEATURE_ENGINEERING();

-- Child tasks chained with AFTER
CREATE OR REPLACE TASK PIPELINE_TRAIN_TASK
AFTER PIPELINE_FEATURE_ENG_TASK
AS CALL RUN_TRAIN();

ALTER TASK PIPELINE_FEATURE_ENG_TASK RESUME;
-- Manual: EXECUTE TASK PIPELINE_FEATURE_ENG_TASK;

```

FAILURE BEHAVIOR

If **EVALUATE** fails the promotion gate, **DEPLOY** and **MONITOR** are **skipped**. The live service continues on the last approved version — no downtime.

TASK OBSERVABILITY

All run history, durations, and errors visible in **Snowsight** → **Data** → **Tasks**. Configure alerts with Slack/email on failure.

CLI MANAGEMENT

```

# Rebuild from scratch
python -m source.pipeline.dag --teardown
python -m source.pipeline.dag --build
# Trigger immediately
python -m source.pipeline.dag --run

```